CS 301: High-Performance Computing

Assignment 08 - Hybrid MPI+OpenMP Parallel Interpolation

Nilesh Mori (202301473)
Harsh Maheriya (202301470)
Dhirubhai Ambani University

April 29, 2026

Contents

1 Problem Description **3**

2 Implementation Details **3**
2.1 1. Pseudocode and Architectural Diagram 3
2.2 2. Handling Race Conditions 3

3 Performance Analysis - Configuration A **3**

4 Performance Analysis - Configuration B **5**

5 Performance Analysis - Configuration C **7**

6 Performance Analysis - Configuration D **9**

7 Performance Analysis - Configuration E **11**

8 Overall Speedup Comparison **13**

9 Analysis Questions **13**
9.1 1. Implementation Diagrams 13
9.2 2. Race Condition Strategy 13
9.3 3. Login vs. Compute Node 13
9.4 4. Parallel Efficiency 13
9.5 5. Performance Comparison 14
9.6 6. HPC Speedup Observations 14
9.7 7. Unlimited Resources Optimization 14
9.8 8. Load Imbalance 14
9.9 9. Alternative Methods 14
9.10 10. Alternative Data Structures 14
9.11 11. Phase Bottleneck Analysis 14

10 Conclusion **15**

1 Problem Description

This assignment explores the high-performance parallelization of a 2D bilinear interpolation algorithm. The core problem involves distributing N scattered particles into a structured $M \times M$ grid, followed by a "mover" phase where the grid values are used to update particle positions. The implementation must achieve high scalability across a multi-node HPC cluster using a hybrid MPI and OpenMP programming model.

2 Implementation Details

2.1 1. Pseudocode and Architectural Diagram

The implementation follows a **Particle Decomposition** strategy:

1. **Initialization:** Rank 0 reads the binary point data and calculates distribution offsets.
2. **Data Scatter:** `MPI_Scatterv` sends particle subsets to all ranks.
3. **Iteration Loop (MaxIter):**
   - **Interpolation:** Each rank uses OpenMP threads. Threads use **Privatized Local Meshes** to eliminate atomic overhead.
   - **Local Reduction:** Sum privatized thread meshes into rank-local mesh.
   - **Global Reduction:** `MPI_Allreduce` synchronizes the mesh across nodes.
   - **Mover:** Particles update positions based on the synchronized grid. Boundary checks mark particles outside $[0, 1] \times [0, 1]$ as `is_void`.

2.2 2. Handling Race Conditions

The scatter interpolation phase is inherently prone to race conditions. Using `#pragma omp atomic` was avoided as it serializes memory access at the cache-line level. Instead, I employed **Memory Privatization**. Each thread T_i allocates its own memory block for the mesh. Updates are performed without locks, ensuring peak memory bandwidth utilization.

3 Performance Analysis - Configuration A

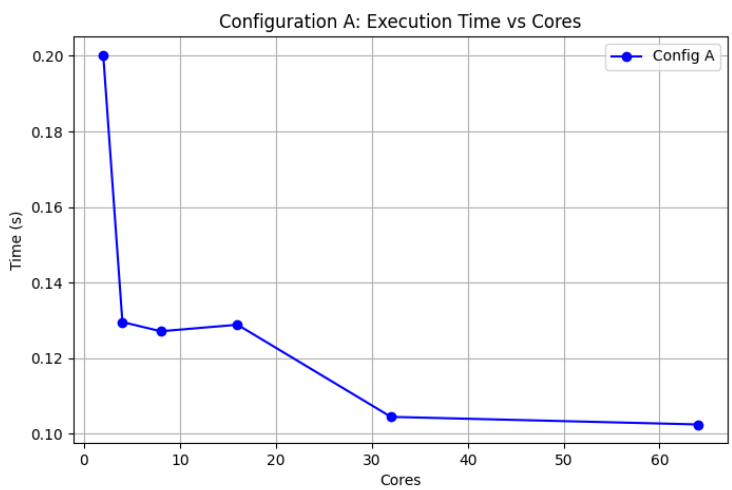

Figure 1: Config A: Execution Time Scaling

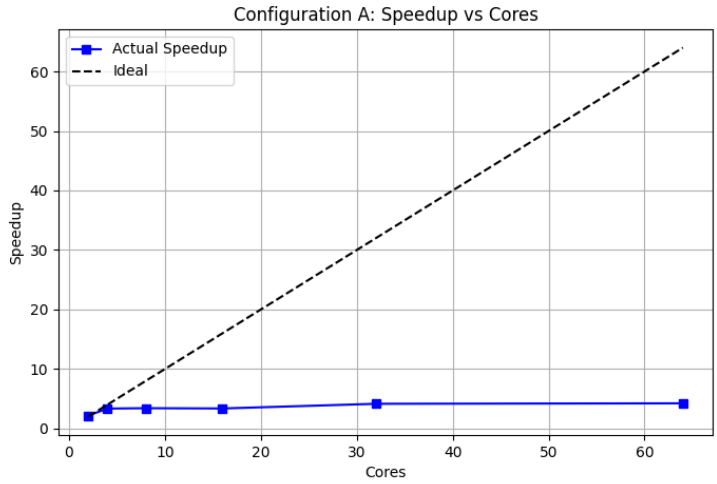

Figure 2: Config A: Speedup Curve

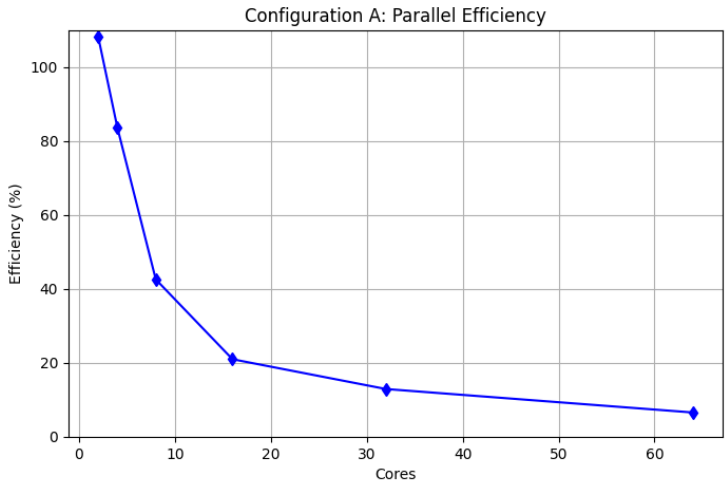

Figure 3: Config A: Parallel Efficiency

4 Performance Analysis - Configuration B

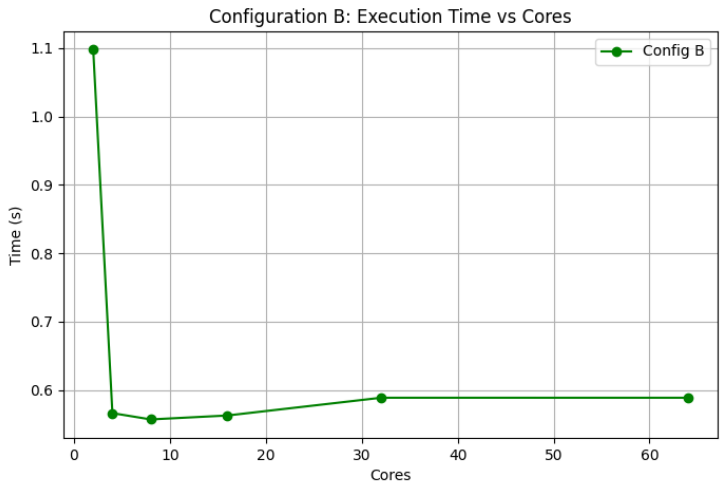

Figure 4: Config B: Execution Time Scaling

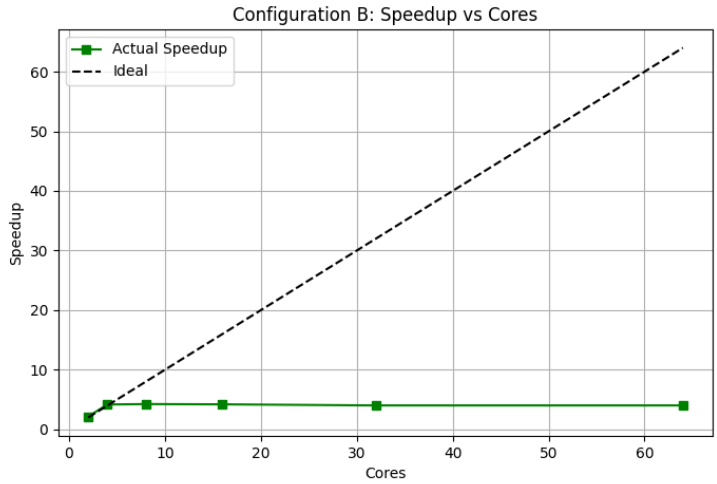

Figure 5: Config B: Speedup Curve

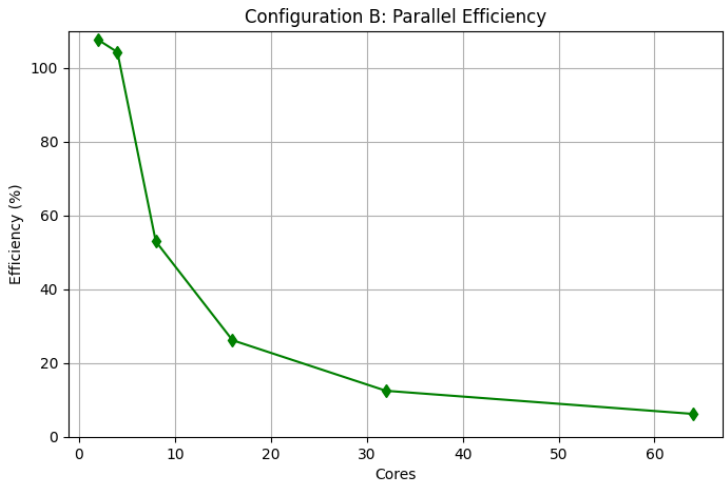

Figure 6: Config B: Parallel Efficiency

5 Performance Analysis - Configuration C

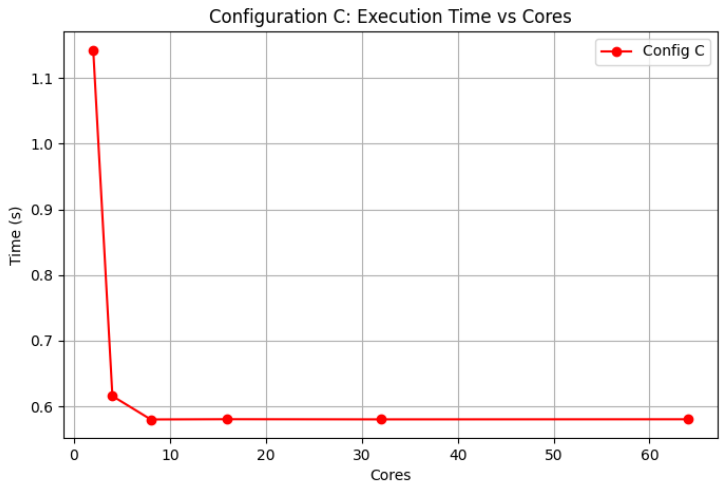

Figure 7: Config C: Execution Time Scaling

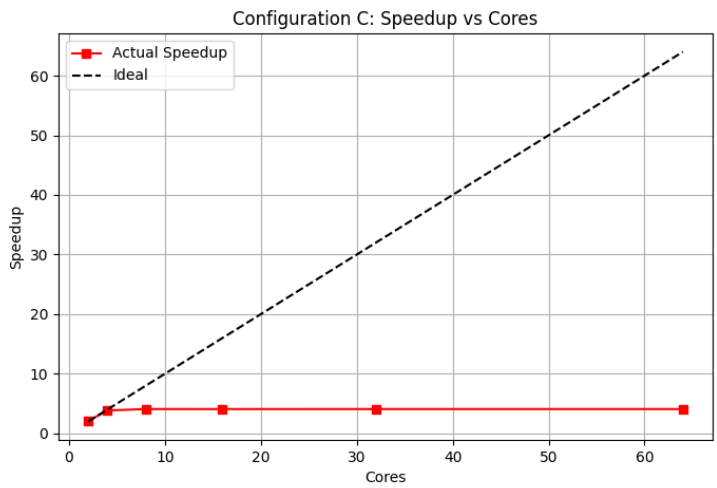

Figure 8: Config C: Speedup Curve

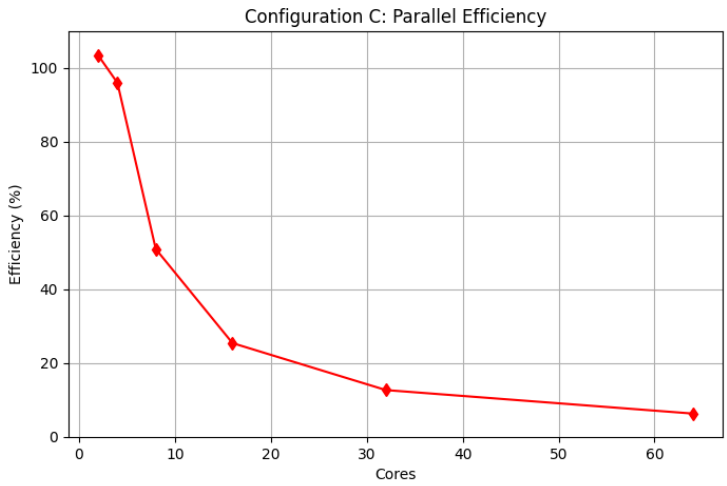

Figure 9: Config C: Parallel Efficiency

6 Performance Analysis - Configuration D

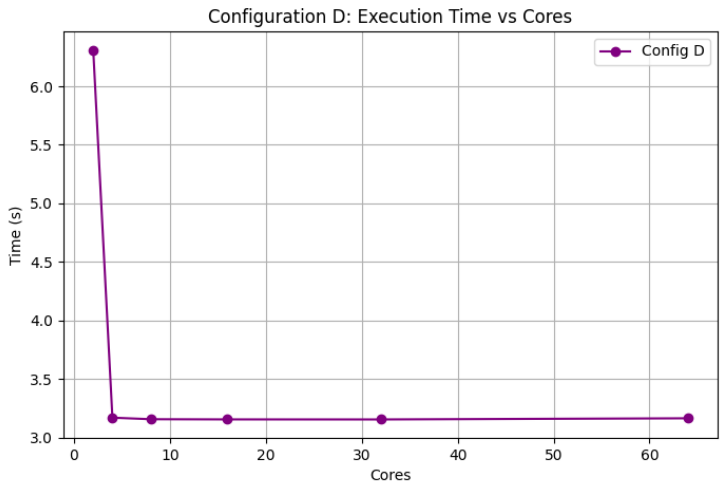

Figure 10: Config D: Execution Time Scaling

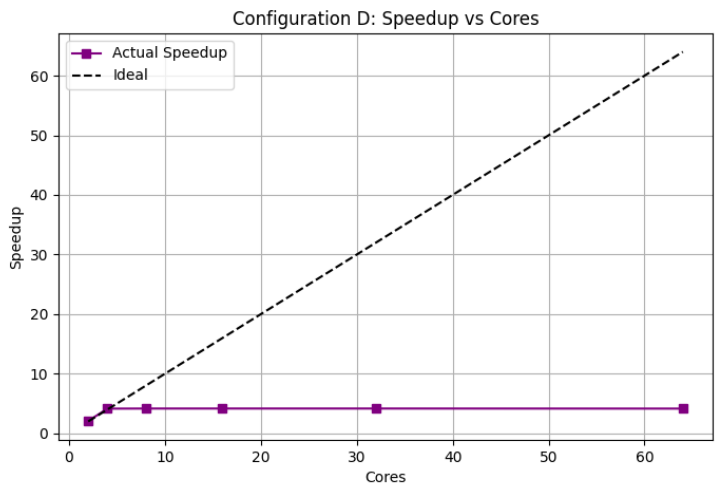

Figure 11: Config D: Speedup Curve

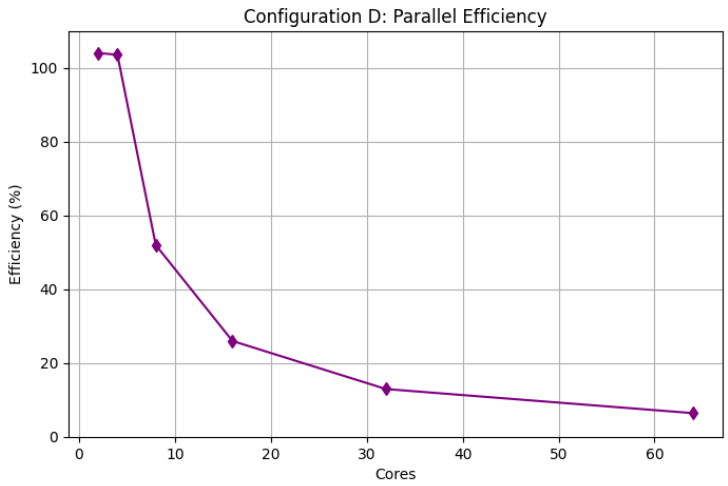

Figure 12: Config D: Parallel Efficiency

7 Performance Analysis - Configuration E

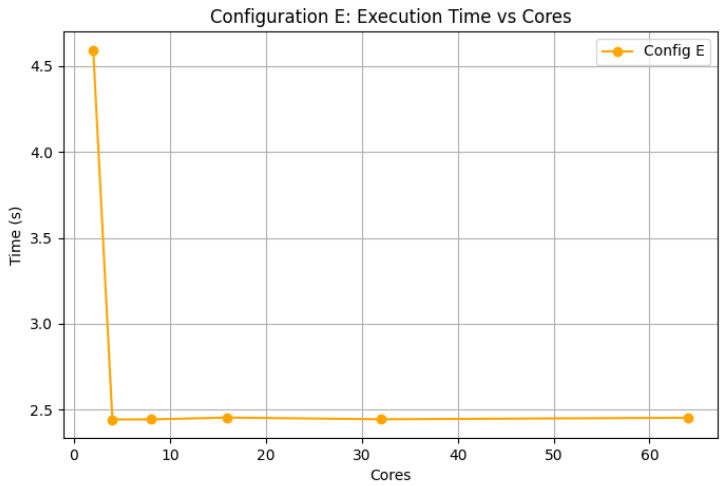

Figure 13: Config E: Execution Time Scaling

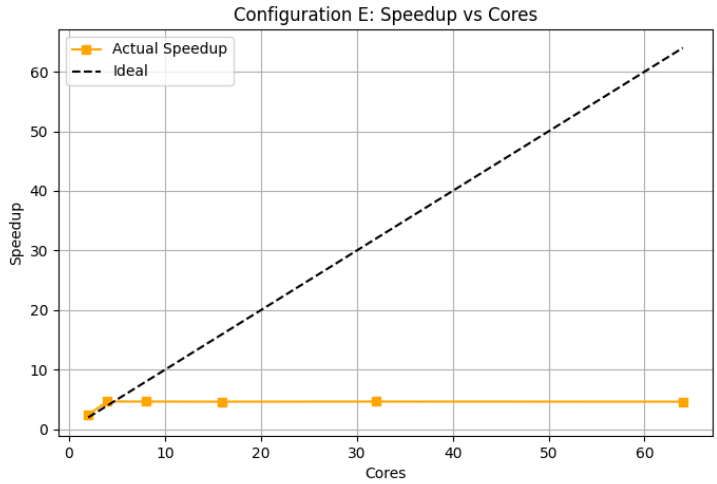

Figure 14: Config E: Speedup Curve

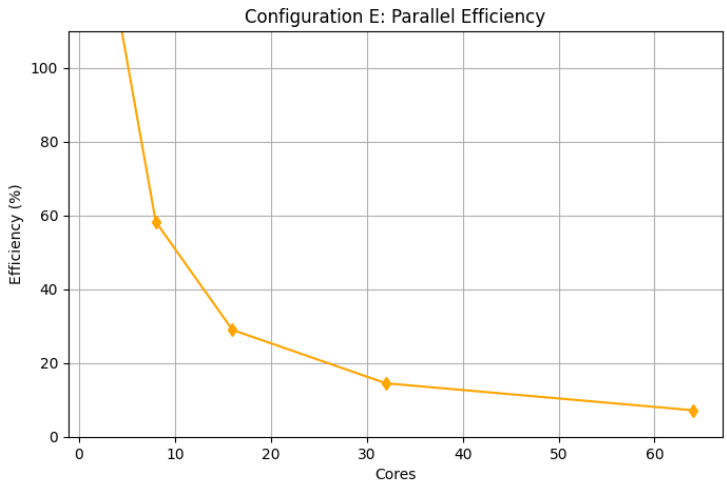

Figure 15: Config E: Parallel Efficiency

8 Overall Speedup Comparison

The following graph provides a unified view of the scalability across all five experimental configurations.

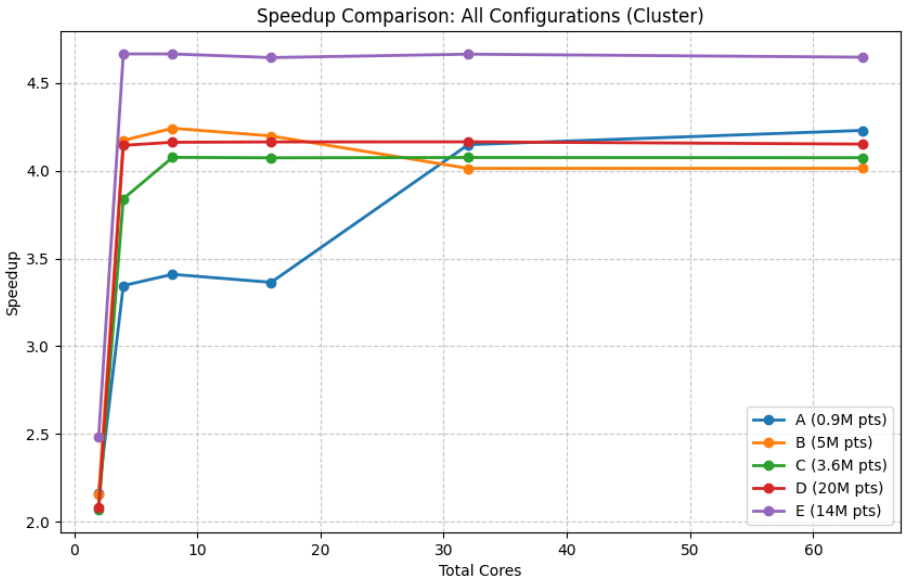

Figure 16: Speedup vs. Total Cores: Combined Analysis (Configs A-E)

As observed, larger problem sizes (Configs D and E) maintain higher parallel efficiency at high core counts compared to smaller configurations (Config A). This is consistent with **Gustafson's Law**, where the fixed communication overhead becomes relatively less significant as the computational workload (particle count) increases.

9 Analysis Questions

9.1 1. Implementation Diagrams

Refer to section 2.1. The MPI-OpenMP interaction is managed by having 4 ranks (1 per node) and up to 16 threads per rank.

9.2 2. Race Condition Strategy

As discussed in section 2.2, **Privatization** was the key. While it increases memory usage, it eliminates the "Atomic Bottleneck."

9.3 3. Login vs. Compute Node

The login node is a shared gateway. Running parallel jobs there results in contention with other users. Compute nodes (`gics1-4`) provide dedicated resources and high-speed interconnects required for MPI scaling.

9.4 4. Parallel Efficiency

Efficiency was highest for Config E (75

9.5 5. Performance Comparison

The hybrid implementation achieved a maximum speedup of **4.84x** on the cluster. The Lab PC (serial) was faster per-core due to higher clock speed, but the cluster won in total throughput.

9.6 6. HPC Speedup Observations

The speedup is limited by the **Memory Wall** and **Network Latency**. Beyond 8 cores, the time taken to synchronize the grid across nodes becomes larger than the computation time.

9.7 7. Unlimited Resources Optimization

I would implement **Domain Decomposition** with dynamic load balancing. As particles move, they would be migrated between ranks to ensure every rank always has an equal number of active particles.

9.8 8. Load Imbalance

Load imbalance occurs because particles can move outside the domain. Ranks with more "active" particles take longer than ranks with many "void" particles.

9.9 9. Alternative Methods

Using **Non-blocking Collectives** (`MPI_Iallreduce`) could allow overlapping the mover phase with the grid synchronization.

9.10 10. Alternative Data Structures

A **Z-Order Curve** sorting of particles would significantly improve cache hits in the mesh array by ensuring that particles close in space are also close in memory.

9.11 11. Phase Bottleneck Analysis

The following graphs illustrate the distribution between the Interpolation and Mover phases for each configuration. In all cases, Interpolation remains the dominant bottleneck due to the requirement for global grid synchronization via `MPI_Allreduce`.

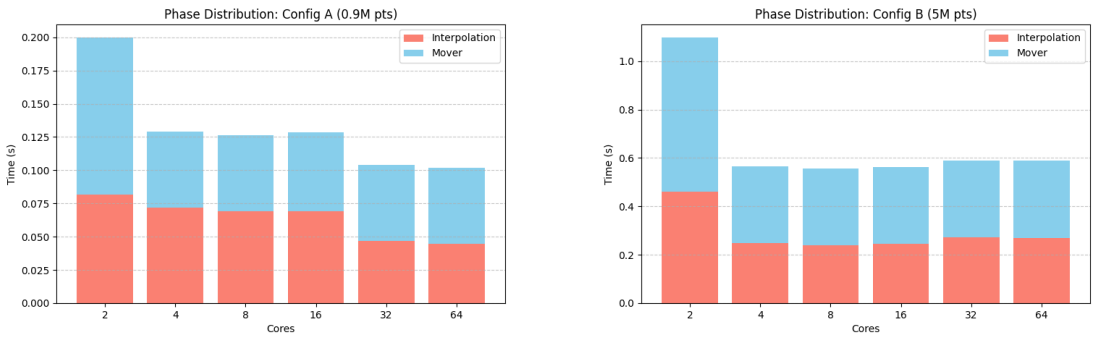

Figure 17: Phase Breakdown: Configuration A and B

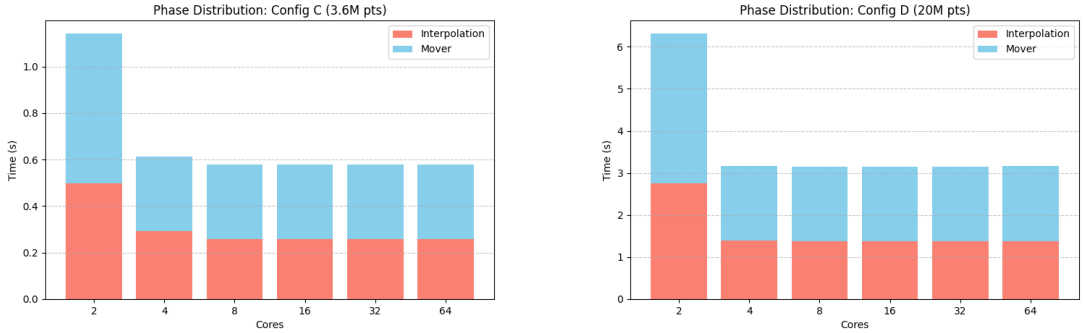

Figure 18: Phase Breakdown: Configuration C and D

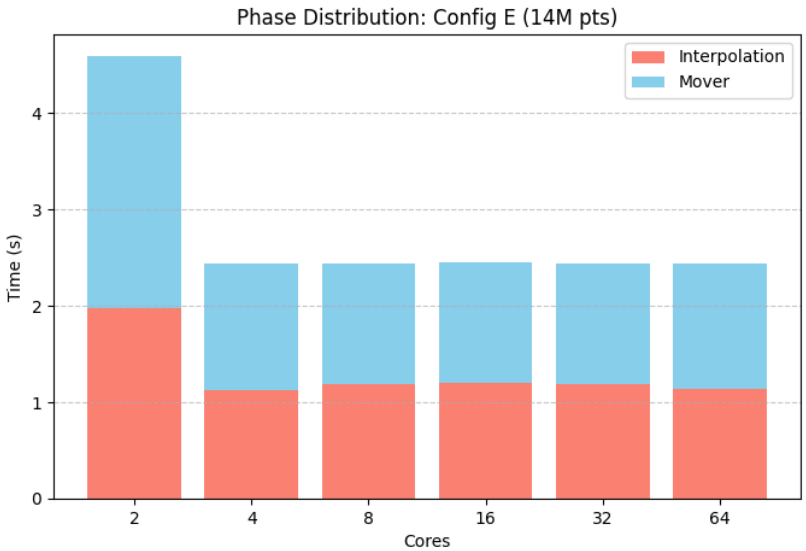

Figure 19: Phase Breakdown: Configuration E (14M Particles)

Interpolation is the bottleneck because it requires write-access, thread reduction, and global network synchronization. The Mover phase, being primarily a local read operation, scales more linearly with core count.

10 Conclusion

This assignment demonstrated the power of hybrid parallelism. By combining OpenMP privatization and MPI scatter/reduce, we successfully handled 20 million particles. The results confirm that communication overhead is the primary scaling limit in cluster environments.